

LTF — Lista de Tarefas

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	LTF — Lista de Tarefas (aplicação web de lista de tarefas)
Gerente de Projeto	José Voss
Gerente/Responsável de GCS-DevOps	José Voss
Fábrica de Software / Squad	SQV — Squad Voss (atividade individual)
Repositório principal	https://github.com/Josevoss7/lista-de-tarefas

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
22/09/2026	1.1	Adaptação do template ao projeto Lista de Tarefas: escopo, ferramentas, ambientes, pipelines, quality gate, CCM, marcos e treinamento.	José Voss

Sumário

1. Introdução.....	3
1.1 Finalidade.....	3
1.2 Escopo.....	3
1.3 Definições, Acrônimos e Abreviações.....	4
1.4 Referências.....	4
1.5 Visão Geral.....	5
2. Cultura DevOps e Papéis na Gerência de Configuração.....	5
3. Ferramentas, Ambientes e Infraestrutura.....	6
3.1 Ferramentas do Pipeline.....	6
3.2 Ambientes e Infraestrutura.....	6
3.3 Estratégia de Branching e Versionamento.....	7
4. Identificação da Configuração.....	8
4.1 Itens de Configuração.....	8
4.2 Convenção de Nomenclatura.....	8
4.3 Estrutura do Repositório.....	8
5. Integração Contínua (CI).....	9
5.1 Etapas do Pipeline de CI.....	9
5.2 Política de Pull Request e Revisão de Código.....	9
5.3 Critérios de Qualidade (Quality Gate).....	9
6. Entrega e Implantação Contínua (CD).....	10
6.1 Etapas do Pipeline de CD.....	10
6.2 Estratégias de Implantação.....	10
6.3 Rollback e Recuperação.....	10
7. Infraestrutura como Código (IaC).....	10
7.1 Versionamento de Infraestrutura.....	11
7.2 Ambientes Efêmeros.....	11
8. Controle de Configuração e Mudança.....	11
8.1 Processo de Solicitação de Mudança.....	11
8.2 Comitê de Controle de Mudança (CCM).....	11
9. Baselines e Releases.....	11
9.1 Geração Automática de Baseline.....	11
9.2 Notas de Versão (Release Notes).....	12
10. Estimativa do Status de Configuração.....	12
10.1 Monitoramento e Observabilidade.....	12
10.2 Métricas DORA.....	12
10.3 Relatórios e Auditorias.....	12
11. Marcos.....	12
12. Treinamento e Recursos.....	13
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros.....	13
14. Quadro-Resumo do Pipeline CI/CD.....	14

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto Lista de Tarefas, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), configuração de ambientes como código e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pelo desenvolvedor, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao estudante José Voss (squad individual SQV), responsável pelo desenvolvimento e operação do sistema Lista de Tarefas, cujo código está no repositório <https://github.com/Josevoss7/lista-de-tarefas>.

A Lista de Tarefas é uma aplicação web simples em que o usuário pode:

- adicionar tarefas (sem título vazio, com no máximo 100 caracteres e sem duplicidade);
- marcar e desmarcar tarefas como concluídas;
- filtrar tarefas em todas, pendentes ou concluídas;
- remover tarefas e limpar as concluídas;
- ver quantas tarefas estão pendentes.

As tarefas ficam salvas no próprio navegador (localStorage). A aplicação é composta apenas por arquivos estáticos (HTML, CSS e JavaScript), sem back-end nem banco de dados — decisão registrada no ADR 0001. A produção é publicada no GitHub Pages e a aplicação também é empacotada como imagem Docker (nginx).



Figura 1 — Tela da aplicação Lista de Tarefas

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) — ex.: 1.0.0
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
PR	Pull Request — solicitação de integração de código na branch principal
ADR	Architecture Decision Record — registro de decisão de arquitetura
GHCR	GitHub Container Registry (ghcr.io) — registro de imagens Docker do GitHub
Smoke test	Teste rápido, após o deploy, que verifica se o sistema está no ar e respondendo

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

- README do projeto e ADRs em /docs/adr do repositório <https://github.com/Josevoss7/lista-de-tarefas>.
- Semantic Versioning 2.0.0 (semver.org) e Conventional Commits 1.0.0 (conventionalcommits.org).
- Documentação do GitHub Actions, GitHub Pages e GitHub Container Registry (docs.github.com).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto DevOps é **responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação**. Por se tratar de atividade individual, todos os papéis abaixo são exercidos por José Voss; a separação é mantida para deixar claro qual papel está sendo exercido em cada atividade.

Papel	Responsabilidades	Responsável
Gerente de Projeto / Product Owner	Prioriza o backlog (GitHub Issues), autoriza mudanças de maior impacto, aprova o gate de produção e acompanha as métricas DORA.	José Voss
Engenheiro DevOps / Release Engineer	Mantém os pipelines de CI/CD (GitHub Actions), a estratégia de branching e versionamento, o registro de imagens (GHCR) e a configuração dos ambientes.	José Voss
Desenvolvedor(a)	Integra código com frequência na main via PR, escreve testes automatizados e é responsável pela qualidade do que é implantado.	José Voss
SRE / Operações	Acompanha os smoke tests e o funcionamento da produção e executa o rollback quando necessário.	José Voss
QA / Automação de Testes	Mantém a suíte de testes automatizados (Jest) e os smoke tests executados nos pipelines de CI e CD.	José Voss
Segurança (DevSecOps)	Mantém a análise estática de segurança (CodeQL), a análise de dependências (npm audit e Dependabot) e garante que não haja segredos no repositório.	José Voss
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ver 8.2); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.	José Voss + professor da disciplina

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas pelo projeto (todas gratuitas):

Categoria	Ferramenta adotada	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, testes, configurações e pipelines como código
Orquestração de CI/CD	GitHub Actions	Automação de build, lint, testes, análise de segurança, implantação e release
Linguagem / aplicação	HTML, CSS, JavaScript	Aplicação web estática
Testes automatizados	Jest	Testes unitários das regras de negócio, com medição de cobertura
Análise estática (lint)	ESLint	Padrões de codificação e detecção de erros
Segurança do código (SAST)	GitHub CodeQL	Identificação de vulnerabilidades no código-fonte
Gestão de dependências (SCA)	npm audit + Dependabot	Vulnerabilidades e atualização automática de bibliotecas de terceiros
Empacotamento	Docker (nginx:1.27-alpine)	Geração de imagem versionada e imutável da aplicação
Registro de artefatos	GitHub Container Registry (ghcr.io)	Armazenamento e versionamento das imagens Docker
Hospedagem de produção	GitHub Pages	Publicação da aplicação para os usuários finais
Versionamento / release	semantic-release	Cálculo automático da versão (SemVer), tag e release notes
Gestão de segredos	GITHUB_TOKEN (automático do Actions)	Credencial temporária usada pelos pipelines; nenhum segredo fica no código
Rastreamento de tarefas	GitHub Issues	Solicitações de mudança vinculadas a branches, commits e PRs

3.2 Ambientes e Infraestrutura

A aplicação não utiliza servidores em nuvem próprios. Os ambientes são definidos por arquivos versionados no repositório (Dockerfile, nginx.conf, docker-compose.yml e os workflows do GitHub Actions), o que garante que possam ser recriados a qualquer momento de forma idêntica e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Execução local pelo desenvolvedor (npm start ou docker compose up) para desenvolver e testar a mudança.	Push na branch de feature e abertura de PR para a main
Homologação / Staging	Container Docker da nova versão criado no runner do GitHub Actions, onde são executados os smoke tests.	Merge na main (pipeline de CI aprovado)

Ambiente	Finalidade	Gatilho de promoção
Produção	GitHub Pages, acessado pelos usuários finais.	Aprovação manual do gate de release (environment github-pages) após sucesso em homologação

Detalhamento por ambiente:

Item	Desenvolvimento	Homologação	Produção
Endereço	http://localhost:8080	http://localhost:8080 dentro do runner (efêmero)	https://josevoss7.github.io/lista-de-tarefas/
Execução	npm start (http-server) ou docker compose	docker run da imagem gerada no pipeline	GitHub Pages (arquivos de /src)
Artefato	Código local	Imagem ghcr.io/josevoss7/lista-de-tarefas:<sha> e :homologacao	Artefato do Pages + imagem :<versão> e :latest
Verificação de saúde	Manual	GET /health e página inicial	GET da página inicial (smoke test)
Credenciais	Nenhuma	GITHUB_TOKEN (temporário)	GITHUB_TOKEN + OIDC do Pages

3.3 Estratégia de Branching e Versionamento

É adotada a estratégia de Trunk-Based Development (no formato GitHub Flow), com branches de vida curta, para viabilizar integração contínua real (ADR 0002):

- main — sempre implantável (deployable); protegida por regra que exige PR e pipeline de CI verde.
- feature/<issue>-<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- fix/<issue>-<descrição-curta> — correções de defeitos seguindo o mesmo fluxo.
- hotfix/<issue>-<descrição> — correções emergenciais em produção (ver CCM, seção 8.2).

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits. O merge de PRs é feito por "Squash and merge", de modo que o título do PR (validado automaticamente pelo workflow pr-titulo.yml) vira a mensagem do commit na main. A partir desses commits, o semantic-release calcula a próxima versão:

Tipo de commit	Efeito na versão	Exemplo
fix:	PATCH	v1.0.0 → v1.0.1
feat:	MINOR	v1.0.1 → v1.1.0
feat!: ou BREAKING CHANGE	MAJOR	v1.1.0 → v2.0.0
docs:, chore:, test:, ci:, refactor:	Não gera nova versão	—

4. Identificação da Configuração

4.1 Itens de Configuração

Os seguintes artefatos são itens de configuração formais, versionados no repositório lista-de-tarefas:

Item de configuração	Arquivo(s)
Código-fonte da aplicação	src/index.html, src/style.css, src/app.js, src/tarefas.js
Testes automatizados	tests/tarefas.test.js
Pipelines como código	.github/workflows/ci.yml, cd.yml, codeql.yml, rollback.yml, pr-titulo.yml
Configuração de container e ambientes	Dockerfile, nginx.conf, docker-compose.yml, .dockerignore
Configuração de qualidade e release	eslint.config.js, .releaserc.json, configuração do Jest no package.json
Dependências (travadas)	package.json e package-lock.json
Automação de dependências	.github/dependabot.yml
Modelos de processo	.github/pull_request_template.md, .github/ISSUE_TEMPLATE/mudanca.md
Documentação técnica	README.md, docs/adr/*.md, docs/PGCS_Lista_de_Tarefas.pdf
Scripts de banco de dados	N/A — a aplicação não possui banco de dados

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Issue	#<número> no GitHub Issues	#1
Branch de feature	feature/<issue>--<descrição-curta>	feature/1-contador-concluidas
Branch de correção	fix/<issue>--<descrição-curta>	fix/4-titulo-com-espacos
Commit / título de PR	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(tarefas): mostra total de concluídas
Tag de release	Versionamento Semântico vX.Y.Z	v1.1.0
Imagem de container	<registro>/<usuário>/<projeto>:<versão sha>	ghcr.io/josevoss7/lista-de-tarefas:1.1.0
ADR	docs/adr/<nnnn>--<título>.md	0001-aplicacao-estatica-sem-backend.md

4.3 Estrutura do Repositório

```

lista-de-tarefas/
├── .github/
│   ├── workflows/      ci.yml, cd.yml, codeql.yml, rollback.yml, pr-titulo.yml
│   ├── ISSUE_TEMPLATE/ mudanca.md
│   ├── dependabot.yml
│   └── pull_request_template.md
├── src/                 index.html, style.css, app.js, tarefas.js
├── tests/               tarefas.test.js
├── docs/                adr/, PGCS_Lista_de_Tarefas.pdf, screenshot.png
├── Dockerfile, nginx.conf, docker-compose.yml
├── eslint.config.js, .releaserc.json
└── package.json, package-lock.json

```

└─ README.md

5. Integração Contínua (CI)

Todo Pull Request para a branch main dispara automaticamente o pipeline de CI (workflow `.github/workflows/ci.yml`). Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso. O mesmo pipeline é reexecutado no início do pipeline de CD, após o merge.

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (npm ci, com cache).
2. Build: preparação do ambiente Node.js 22 e das dependências da aplicação.
3. Análise estática de código (npm run lint — ESLint).
4. Testes unitários (npm run test:coverage — Jest), com cobertura mínima de 80%.
5. Publicação do relatório de cobertura como artefato do pipeline.
6. Análise de segurança do código (CodeQL — SAST, workflow `codeql.yml`) e de dependências (npm audit — SCA).
7. Build da imagem Docker para validar o empacotamento (a publicação no registro ocorre no CD).

5.2 Política de Pull Request e Revisão de Código

- Todo PR segue o modelo `pull_request_template.md`, com checklist de revisão. Como a atividade é individual, a revisão "quatro olhos" é feita pelo próprio autor com o checklist e, em mudanças que acionam o CCM, também pelo professor.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate), garantido por regra de proteção da branch main.
- Todo PR referencia a issue correspondente (ex.: "Closes #1") e tem título em Conventional Commits.

5.3 Critérios de Qualidade (Quality Gate)

O PR só pode ser integrado se todos os critérios abaixo forem atendidos (verificados automaticamente):

Critério	Limite
Lint (ESLint)	Zero erros
Testes (Jest)	100% dos testes passando
Cobertura de testes	≥ 80% de linhas, ramos, funções e instruções (coverageThreshold)
Dependências (npm audit)	Zero vulnerabilidades altas ou críticas
Segurança do código (CodeQL)	Nenhum alerta novo de severidade alta ou crítica
Empacotamento	Build da imagem Docker sem erros
Título do PR	No padrão Conventional Commits

6. Entrega e Implantação Contínua (CD)

Após o merge na main, o pipeline de CD (workflow `.github/workflows/cd.yml`) reexecuta o CI e promove a mesma versão, sem alterações, por homologação e produção.

6.1 Etapas do Pipeline de CD

8. Deploy automático em Homologação: build da imagem e execução do container no runner.
9. Smoke tests em homologação (`/health`, página inicial e script da aplicação) e publicação da imagem no GHCR com a tag do commit (sha).
10. Gate de aprovação para produção: aprovação manual no environment github-pages (Required reviewers).
11. Deploy em produção no GitHub Pages.
12. Smoke test pós-deploy na URL de produção.
13. Geração automática de baseline: tag `vX.Y.Z`, GitHub Release com notas de versão e imagem Docker marcada com a versão e com latest.

6.2 Estratégias de Implantação

Estratégia	Descrição	Uso no projeto
Substituição atômica	O GitHub Pages troca a versão publicada de uma só vez para a nova versão, sem estado intermediário (semelhante ao Blue-Green).	Adotada
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade.	Não se aplica (sem servidores próprios)
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança.	Não adotada
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy.	Não adotada nesta versão

6.3 Rollback e Recuperação

Toda implantação é reversível. As versões anteriores permanecem disponíveis como tags no Git e como imagens no GHCR. Se um smoke test falhar, o pipeline é interrompido e a versão anterior continua no ar. Para voltar a uma versão já publicada, executa-se o workflow `rollback.yml` (Actions → Rollback → Run workflow), informando a tag estável (ex.: `v1.0.0`); o workflow publica novamente aquela versão no GitHub Pages.

7. Infraestrutura como Código (IaC)

O projeto não provisiona servidores em nuvem. Mesmo assim, toda a configuração de execução é descrita como código versionado e segue o mesmo fluxo de Pull Request e pipeline usado para a aplicação:

- `Dockerfile` e `nginx.conf` — definem o servidor web, a porta e o endpoint `/health`.
- `docker-compose.yml` — define o ambiente de desenvolvimento local.

- Workflows do GitHub Actions — definem os ambientes de homologação e produção e as permissões de cada etapa.

7.1 Versionamento de Infraestrutura

- Alterações nesses arquivos seguem o mesmo processo de PR + revisão + pipeline; o CI valida o build da imagem antes do merge.
- Não há arquivo de estado (state file), pois não há recursos de nuvem provisionados. Caso o projeto passe a usar nuvem, será adotado Terraform com estado remoto, registrado em novo ADR.

7.2 Ambientes Efêmeros

O ambiente de homologação é efêmero: a cada execução do pipeline de CD, um container novo é criado no runner do GitHub Actions a partir da imagem recém-gerada, testado e destruído automaticamente ao fim do job.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de issue no GitHub (modelo "Solicitação de mudança") → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança (issue ↔ branch ↔ PR ↔ execução do pipeline ↔ tag).

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado. Em situações rotineiras, a revisão do PR substitui o comitê formal.

Composição: José Voss (Product Owner / Engenheiro DevOps) e o professor da disciplina.

Critérios objetivos de acionamento:

- Nova versão MAJOR (mudança que quebra compatibilidade, ex.: formato dos dados salvos no navegador).
- Alteração dos pipelines que remova ou enfraqueça etapas do quality gate.
- Hotfix aplicado fora do fluxo padrão (analisado em até 48 horas após o deploy).
- Alteração das regras de proteção da branch main ou do gate de produção.

A decisão do CCM é registrada como comentário na issue correspondente, com o rótulo "ccm-aprovado".

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão SemVer (ex.: v1.0.0) associada ao commit, à imagem publicada no registro (ghcr.io/josevoss7/lista-de-

tarefas:1.0.0) e ao artefato publicado no GitHub Pages. Commits que não geram versão (docs, chore, ci etc.) são implantados, mas não criam nova baseline.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo semantic-release a partir dos Conventional Commits desde a última tag, e publicadas na aba Releases do repositório.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Por ser uma aplicação estática, o monitoramento é feito pelos próprios pipelines: smoke tests em homologação e produção, verificação de saúde do container (HEALTHCHECK do Dockerfile e endpoint /health), notificações de falha de workflow do GitHub por e-mail e o painel da aba Actions.

10.2 Métricas DORA

Métrica	Como é medida no projeto	Meta
Deployment Frequency	Número de execuções bem-sucedidas do pipeline de CD (aba Actions / Deployments).	Ao menos 1 deploy por semana
Lead Time for Changes	Tempo entre o primeiro commit do PR e o deploy em produção.	Menos de 1 dia
Change Failure Rate	Percentual de deploys que exigiram rollback ou hotfix.	Até 15%
Mean Time to Restore (MTTR)	Tempo entre a falha detectada e a execução do rollback.	Menos de 1 hora

10.3 Relatórios e Auditorias

O histórico de execuções do GitHub Actions (builds, testes, deploys), o histórico do Git, os PRs, as tags e as releases constituem o principal insumo de auditoria de GCS, dispensando relatórios manuais.

11. Marcos

Marco	Descrição	Data
M1	Repositório criado, convenções definidas e regras de proteção da main configuradas	22/09/2026
M2	Pipeline de CI implantado (lint, testes, cobertura, SAST/SCA, build Docker)	22/09/2026
M3	Pipeline de CD implantado (homologação, gate de aprovação, produção no GitHub Pages)	23/09/2026
M4	Primeira release automatizada em produção (v1.0.0)	23/09/2026
M5	Primeira mudança completa pelo fluxo issue → PR → CD (v1.1.0)	23/09/2026
M6	Verificação do PGCS e do repositório em laboratório	Conforme ordem no Teams

12. Treinamento e Recursos

Pessoal: 1 pessoa (José Voss), exercendo os papéis descritos na seção 2.

Treinamentos:

- Git e GitHub (branches, Pull Requests, Issues) — GitHub Skills.
- GitHub Actions — documentação oficial e trilha do GitHub Skills.
- Docker — conceitos de imagem, container e Dockerfile.
- Testes automatizados com Jest.
- Conventional Commits e Versionamento Semântico.

Recursos: conta gratuita no GitHub (repositório público, GitHub Actions, Pages e Container Registry sem custo), computador com Git, Node.js 22 e, opcionalmente, Docker Desktop.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Não há software desenvolvido por subcontratados ou fornecedores. As bibliotecas e imagens de terceiros são itens de configuração e são versionadas explicitamente:

Dependência	Uso	Controle
jest, eslint, @eslint/js, globals	Testes e lint (desenvolvimento)	Versões travadas no package-lock.json
semantic-release	Geração de versão e release	Versão travada no package-lock.json
nginx:1.27-alpine	Imagem base do container	Versão fixada no Dockerfile (nunca "latest")
Actions de terceiros	Etapas dos workflows	Versão fixada (ex.: actions/checkout@v4)

- Análise automática de vulnerabilidades em dependências (npm audit) integrada ao pipeline de CI.
- Atualizações de npm, GitHub Actions e imagem Docker propostas semanalmente pelo Dependabot e submetidas ao mesmo fluxo de PR.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Lint, testes com cobertura, npm audit, CodeQL e build da imagem Docker.	Abertura/atualização de Pull Request
Merge	Squash and merge da mudança na branch main.	CI verde + revisão do PR
CD — Homologação	Container da nova versão, smoke tests e publicação da imagem no GHCR.	Merge na branch main
Gate de Release	Aprovação manual para produção pelo Product Owner.	Sucesso em homologação
CD — Produção	Deploy no GitHub Pages, smoke test e geração de baseline (tag, release, imagem versionada).	Aprovação do gate de release

Nota: este plano foi adaptado ao contexto real do projeto (aplicação estática, atividade individual e ferramentas gratuitas do GitHub), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — suportados por automação.